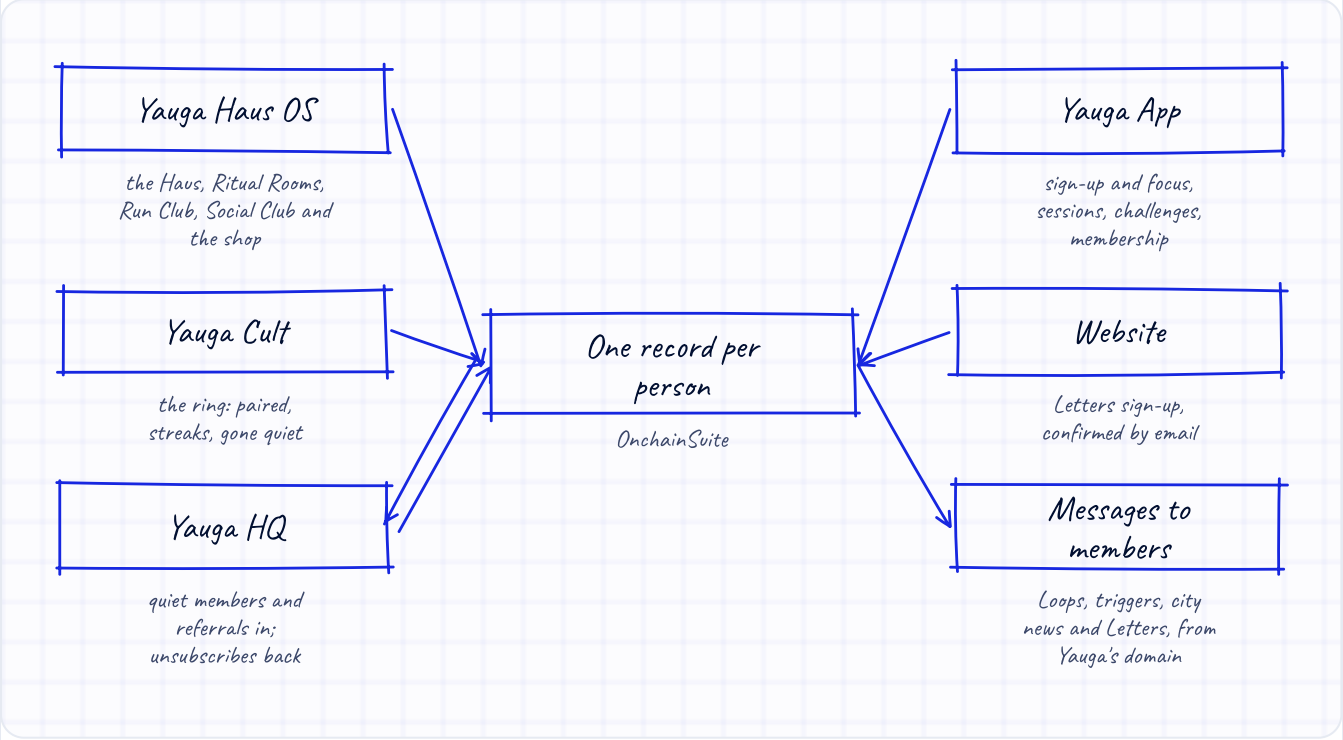
Yauga growth and retention strategy

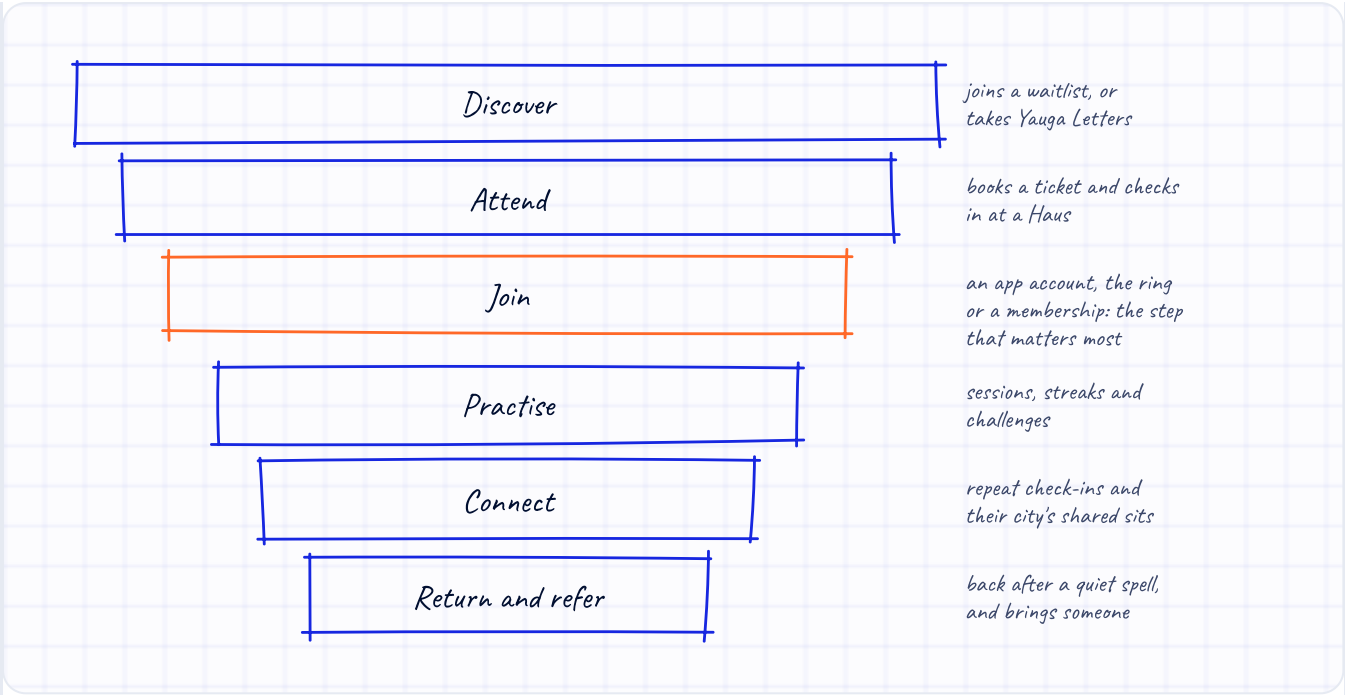
How communication follows a member from a city's waitlist to the Haus, the app and the ring, and brings them back when they go quiet.

Today Yauga's systems do not know each other: Haus emails go out from Haus OS and Yauga Letters from OnchainSuite, while the app, Yauga Cult, the shop, Run Club and Social Club send nothing at all, so a person who goes to a Haus and then joins the app is two strangers to the way Yauga communicates.

We propose one record per person, so every Yauga system reports what happened and communication follows the person rather than the product. The step that matters most is Attend to Join, because live experiences bring people in and email is how they find the app and the ring after they go home, so the first automations we build start on a check-in.

The plan has four parts, campaigns and the newsletter, Loops, lifecycle and retention triggers, all carried by email from Yauga's own domain, with one broadcast a week and every email passing the Yauga Test.





What we never send

Body data stays with Yauga. Sleep, heart rate, HRV, stress and SpO_2 readings never leave Yauga's systems, and OnchainSuite only hears that something happened, such as a seven-day streak, which matches "your data is yours" and keeps health data out of member communication altogether.

The Yauga rules

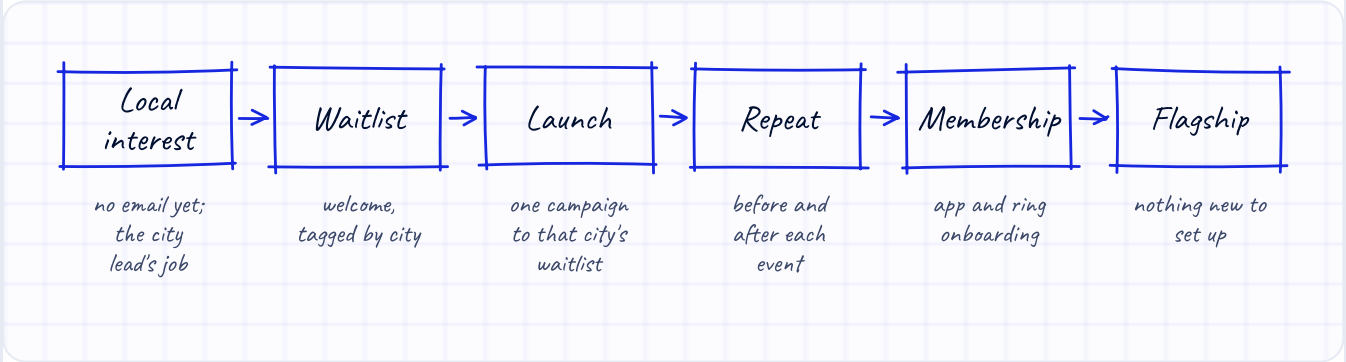
- Body data never leaves Yauga's systems; OnchainSuite only hears that something happened
- No marketing to anyone who has not said yes: a ticket buyer gets their ticket, and every automation checks for the letters tag
- One broadcast a week, and since OnchainSuite has no built-in limit on how often one person is emailed, that rule is the limit
- Nothing that is not secular, and anything at 40 or above on the woo-woo scale is rewritten or cut
- City leads never send their own emails, which would split the voice and the sending reputation, and receipts stay in Haus OS, where they already work

Everything we do send passes the Yauga Test, which asks whether it can be felt, practised and explained plainly. Every moment has one owner, so nobody gets two emails about the same thing, every member email comes from one sender with one standard in every city, and practitioners, city leads and partners sit on a separate list, never mixed into member sends.

Campaigns and the newsletter

A campaign goes to a tag, because something is happening for the people who carry it, and since a city is only a tag, the same system runs every city from one workspace. A member who moves from Bournemouth to Barcelona stays one person with one history, and Chicago and Vienna get separate sends in their own time.

Send	Goes to	When and how
City launch	That city's waitlist	One campaign, when the city opens
Before an event	The event's ticket tag	What to bring and how to arrive ready, unless Haus OS sends a reminder two days out
City news	The city tag	Written by the city lead and sent by HQ, from the same sender as everything else
Yauga Letters	Everyone with the letters tag	Once a week, one broadcast split by focus and city



Yauga Letters carries the member's pillar, whether that is sleep and Tranquility, stress and Stillness, attention and Attention, or movement and Energy, beside what is on in their city. Selling to employers, universities and studios stays outside OnchainSuite, because that is sales rather than member email, and members who join through them get a source tag.

Loops

A Loop is a short sequence of emails to one member. It starts from something they did, or did not do, and stops the moment they take the step it asks for, so nobody is chased for something they have already done.

Welcome

Starts when	What it sends	Stops when
Someone joins a city's waitlist or Yauga Letters through an OnchainSuite form and confirms by email	One email on what they will get, pointing at their city's next event or their place on the waitlist	On confirmation, and nothing goes to an address that has not confirmed

After the Haus

Starts when	What it sends	Stops when
A check-in at a Haus	That evening, one email pointing at the app with a first practice for their focus	They create an account

First practice

Starts when	What it sends	Stops when
An app account is created	A first practice for their pillar, then how to set up the ring	They complete a session or pair the ring

Slipping

Starts when	What it sends	Stops when
Yauga sends member_quiet as the member moves to At risk	The next shared sit in their city, with the date, and nothing else	They check in or complete a session

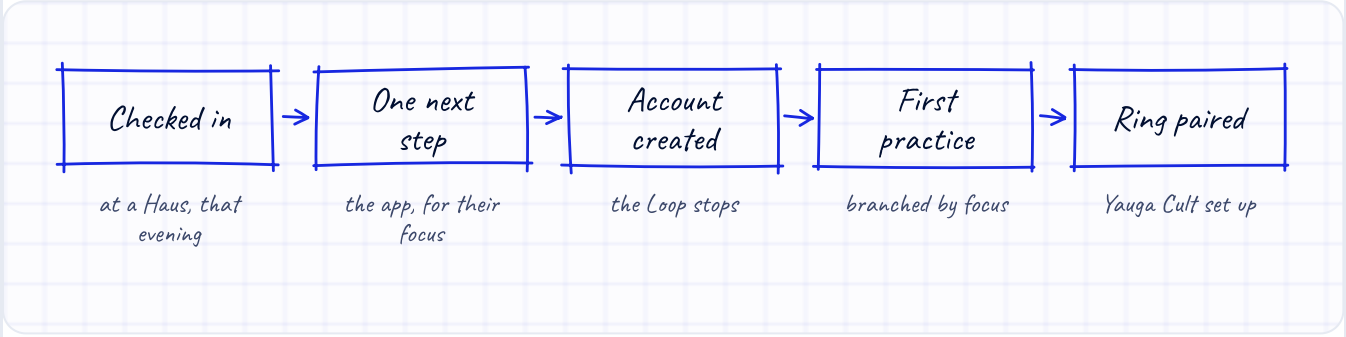
Bring someone

Starts when	What it sends	Stops when

A third check-in, when a member is a regular	→ One email inviting them to bring someone	→ A referral is made

Dormant

Starts when	What it sends	Stops when
60 days without a check-in, a session or a streak	→ One good reason to come back	→ After that one email, and then we leave them alone



Live experiences bring people in. This is how they find the app after they go home.

How every Loop behaves

It starts from a fresh event each time, because on OnchainSuite a member enters a "joined a segment" flow only once, ever

It stops the moment the member takes the step it asks for

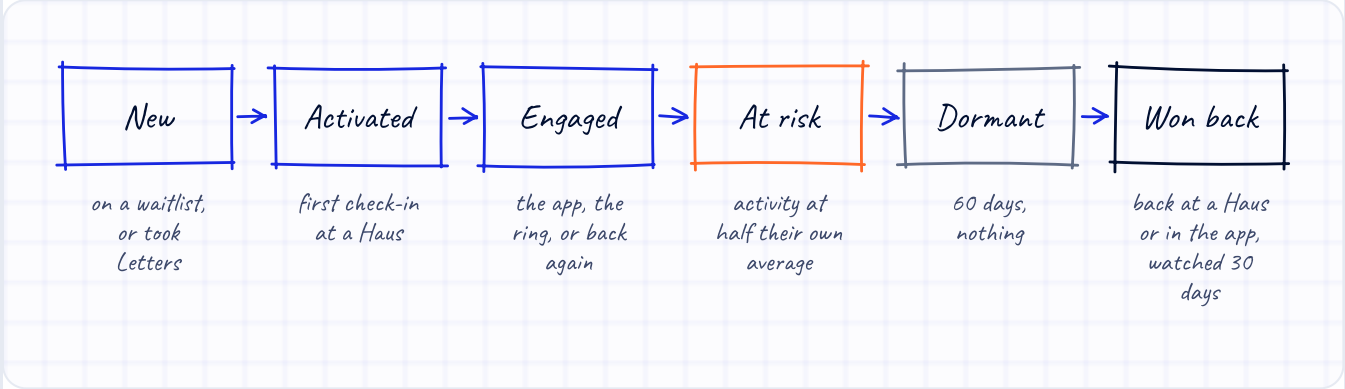
One owner per moment, so nobody gets two emails about the same thing

Email is the only member channel: OnchainSuite's in-app messages and push need a wallet sign-in, which Yauga's members do not use, so the Yauga App keeps its own push

One broadcast a week is the cap, and retention triggers are the exception

Lifecycle

Every member sits in one of six stages, the same six on every strategy we run. The stage is worked out from what the member did across every Yauga system, never set by hand, and it is read by city and by focus; it moves on patterns, while retention triggers answer single events, and the two never overlap.



Stage	What it means	What moves them on
New	On a waitlist or taking Letters, with no check-in yet	The launch campaign and the welcome Loop
Activated	A first check-in at a Haus	The after-the-Haus Loop
Engaged	An app account, a paired ring, a membership or a second check-in	No Loop: Letters and the city's campaigns
At risk	Activity at half their own 30-day average	The slipping Loop
Dormant	60 days with nothing	One email, then rest
Won back	Back at a Haus or in the app from At risk or Dormant	Watched for 30 days, then Engaged or At risk, never back through the welcome

The one rule for At risk

Activity at half the member's own 30-day average, counted across check-ins, sessions and streaks. OnchainSuite's built-in scores are designed for wallets, so Yauga works it out from its own data and sends it as member_quiet.

Each capture point sets what we may send. The website forms, and an opt-in tick at a Haus OS checkout or at app sign-up, carry news consent confirmed by email, while a ticket bought without the tick carries service only, so that person gets their ticket and event follow-ups but never Letters, launches or city news. Where nobody can show what the people on Yauga's existing Haus list agreed to, the first email asks.

Exit points

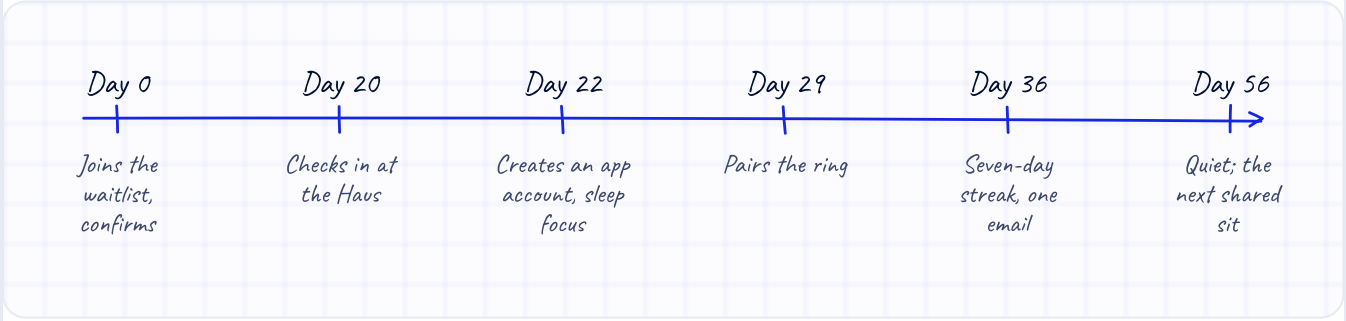
An unsubscribe stops Letters, launches, city news and every Loop, and reaches HQ Control by webhook the same day; tickets and receipts still come from Haus OS

A spam complaint stops everything that day, and a hard bounce closes the address

A cancelled membership is a retention trigger, not an exit

An account deleted, or an erasure request, ends everything, the record included

Take Ada, who joins the Vienna waitlist on the website and confirms with one click, and follow her from there.



Retention triggers

A retention trigger is one email, sent the same day something happens to a member's own account. It comes from the event rather than a calendar, carries no offer, and goes to anyone the event belongs to, with or without the letters tag, because it is about something they did.

Moment	The email
A streak reached or a challenge completed	Notices it and offers one small next practice; Yauga works out the streak and sends only the milestone
The ring gone quiet	Says it has stopped syncing, and how to get it going again
A membership cancelled	What changes, what stays and how to come back, with no offer
Two days before an event	Only if Haus OS sends reminder_due; otherwise the ticket campaign carries it
A refund	Nothing from OnchainSuite, because Haus OS sends the receipt instantly

Here is the after-the-Haus email, the one every other automation depends on, written to pass the Yauga Test.

After a first check-in, email

Subject **What to do with tonight**

Preview One practice for sleep, in the app, before you go to bed

You were at Yauga Haus Vienna tonight. Thank you for coming.

What you felt in the room can be practised at home. The Yauga App has a first practice for sleep that takes eleven minutes, and it works best tonight, while the evening is still with you.

Create your account with the same email and the practice is waiting for you.

Open the app

How we know it worked

Bookings, check-ins, accounts and sessions are counted from Yauga's own events rather than from the estimates in OnchainSuite's automation reports, and every always-on Loop keeps a holdout, a small group left out on purpose, so we can show what the emails caused.

> Opens are a health check. The goal is movement round the loop.

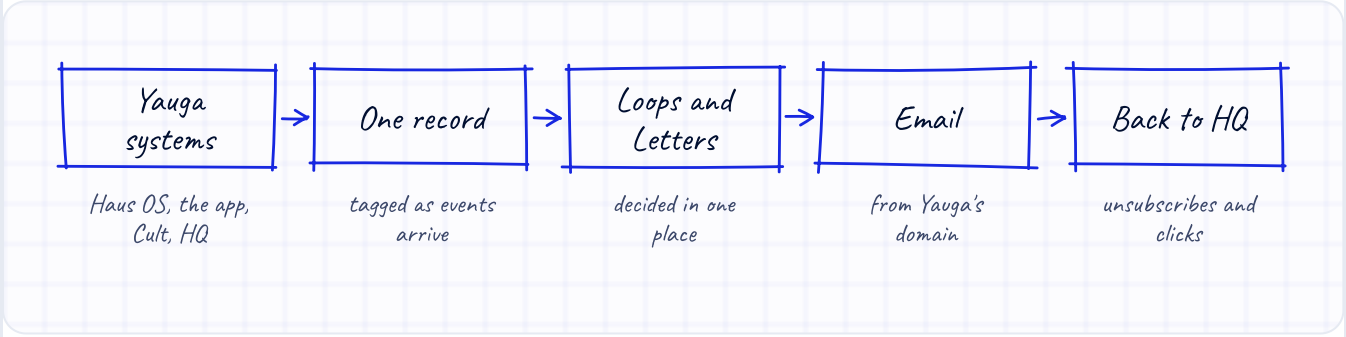
Every Monday a stage report shows how many members sit at each stage, by city and by focus, and the stage where most of them are stuck is where the next week's effort goes. Under it sit five numbers, with targets set once the first month gives us a baseline:

- The share of a city's waitlist who book its first event
- The share of first-time attendees who create an app account
- The share of new app members who complete a first session
- Repeat bookings per member
- Unsubscribes and spam complaints, which should stay low and flat

Running it on OnchainSuite

Every Yauga system sends its events straight to OnchainSuite from Yauga's server, with a secret key. An event arrives in seconds, so a follow-up can go the same day as a check-in, and each system is wired once, after which new emails are built in OnchainSuite with no further developer work.

System	What it reports
Haus OS	A waitlist joined, a ticket booked, a check-in, a refund and an order
Yauga App	An account created, a session or challenge completed, a membership started or cancelled
Yauga Cult	The ring paired, a streak reached and the ring inactive
Yauga HQ	A member gone quiet and a referral made, worked out from Yauga's own data
Website	Opt-ins, through an OnchainSuite form with double opt-in
Back to HQ Control	Unsubscribes and clicks, by webhook



One record per person is found by email first and Yauga member ID second, and tagged as events arrive with who they are, their city, their focus and pillar, the formats they use and their state. Tags keep a group such as "Vienna, sleep focus" current without anyone rebuilding it, and groups that depend on history are built as SQL segments from the event log.

What the platform does today	So the plan
A member enters a "joined a segment" flow once, ever	Repeating moments start from fresh events
An automation cannot wait until two days before an event date	Pre-event emails go as a campaign to the event's ticket tag
The segment builder reads tags, a score and a wallet	Meaning goes into tags at the source, and history uses SQL segments
The engagement and churn scores are designed for wallets	"Quiet" comes from Yauga as member_quiet
Date triggers run on one team time zone	City sends are scheduled separately
Conversion figures in automation reports are estimates	Outcomes are counted from Yauga's events

We chose direct events because they are live and each system stands on its own. Haus OS could forward the app's and the ring's events if Yauga's developers prefer one pipe, at the cost of both depending on Haus OS, while nightly files arrive too late to start a flow and events older than seven days are refused, so an import is for existing history only.

The plan is sized to contacts, Launch at 2,500 and Growth at 25,000, and OnchainSuite brings the event list and tag scheme, the segments and automations built and tested, Yauga Letters structured by focus and city, reporting on the loop, and a team running it alongside Yauga's.

What we need from Yauga

Six things before anything is wired:

> One member ID shared by Haus OS and the app, because otherwise email is the only way to join a person's record together

- A developer to connect each system once
- An opt-in tick at checkout and at app sign-up
- City and focus values sent the same way from every system
- HQ Control listening for unsubscribes
- The city leads' local news sent to HQ, never out from the city

Then, before anything goes out:

- The event names confirmed by Asif, since they are a working draft
- The spelling of "Yauga Cult" confirmed with Kris
- The sending domain and who manages its DNS, because SPF, DKIM and DMARC decide between inbox and spam
- Who approves each email, and whose voice it goes out in

This is the strategy, and copy and timing come after Yauga agrees the approach.